

```

1  import numpy as np
2  import matplotlib.pyplot as plt
3  from scipy import signal
4
5  np.set_printoptions(precision=4, suppress=True)
6  print('NumPy:', np.__version__)
7
8
9  n = np.arange(-6, 7)
10 delta = (n == 0).astype(float)
11 step = (n >= 0).astype(float)
12
13 fig = plt.figure(figsize=(8, 3))
14 plt.stem(n, delta)
15 plt.xlabel('n')
16 plt.ylabel(r'$\delta[n]$')
17 plt.title('Unit sample')
18 plt.grid(True, alpha=0.25)
19 plt.show()
20
21 fig = plt.figure(figsize=(8, 3))
22 plt.stem(n, step)
23 plt.xlabel('n')
24 plt.ylabel(r'$u[n]$')
25 plt.title('Unit step')
26 plt.grid(True, alpha=0.25)
27 plt.show()
28
29
30 # A finite sequence indexed from n=-2 to n=2
31 n_x = np.arange(-2, 3)
32 x = np.array([1.0, -0.5, 2.0, 0.0, 1.5])
33
34 # Reconstruct x[n] by adding scaled shifted impulses.
35 reconstructed = np.zeros_like(x)
36 for k, xk in enumerate(x):
37     reconstructed += xk * (np.arange(len(x)) == k)
38
39 print('x          =', x)
40 print('reconstructed =', reconstructed)
41 print('Exact match?  =', np.allclose(x, reconstructed))
42
43
44 def convolve_by_definition(x: np.ndarray, h: np.ndarray) -> np.ndarray:
45     """Full linear convolution for 1-D finite sequences."""
46     x = np.asarray(x, dtype=float)
47     h = np.asarray(h, dtype=float)
48     y = np.zeros(len(x) + len(h) - 1)
49     for n in range(len(y)):
50         for k in range(len(x)):
51             h_index = n - k
52             if 0 <= h_index < len(h):
53                 y[n] += x[k] * h[h_index]
54     return y
55
56 x = np.array([1.0, 2.0, 1.0])
57 h = np.array([1.0, -1.0])
58
59 y_manual = convolve_by_definition(x, h)
60 y_numpy = np.convolve(x, h, mode='full')
61
62 print('manual:', y_manual)
63 print('numpy :', y_numpy)
64 print('match :', np.allclose(y_manual, y_numpy))
65
66
67 fig = plt.figure(figsize=(8, 3))

```

```

68 plt.stem(np.arange(len(y_numpy)), y_numpy)
69 plt.xlabel('n')
70 plt.ylabel('y[n]')
71 plt.title('Convolution result')
72 plt.grid(True, alpha=0.25)
73 plt.show()
74
75
76 def test_linearity(T, x1, x2, a=1.7, b=-0.4, atol=1e-10):
77     left = T(a*x1 + b*x2)
78     right = a*T(x1) + b*T(x2)
79     return np.allclose(left, right, atol=atol)
80
81 rng = np.random.default_rng(2)
82 x1 = rng.normal(size=10)
83 x2 = rng.normal(size=10)
84
85 T_gain = lambda x: 3*x
86 T_square = lambda x: x**2
87 T_offset = lambda x: x + 2
88
89 print('Gain system linear? ', test_linearity(T_gain, x1, x2))
90 print('Square system linear? ', test_linearity(T_square, x1, x2))
91 print('Offset system linear? ', test_linearity(T_offset, x1, x2))
92
93
94 fs = 8000
95 N = 400
96 n = np.arange(N)
97
98 # Two tones plus reproducible noise
99 x = np.sin(2*np.pi*300*n/fs) + 0.45*np.sin(2*np.pi*2200*n/fs)
100 x += 0.25*np.random.default_rng(3).normal(size=N)
101
102 h = np.ones(5)/5
103 y = np.convolve(x, h, mode='same')
104
105 fig = plt.figure(figsize=(9, 3))
106 plt.plot(n[:120], x[:120], label='input')
107 plt.plot(n[:120], y[:120], label='5-point moving average')
108 plt.xlabel('n')
109 plt.ylabel('amplitude')
110 plt.title('Short segment before and after FIR smoothing')
111 plt.legend()
112 plt.grid(True, alpha=0.25)
113 plt.show()
114
115
116 try:
117     import torch
118     import torch.nn.functional as F
119
120     x_t = torch.tensor([[[[1., 2., 1., 0.]])
121     w_t = torch.tensor([[[[1., -1.]])
122     y_corr = F.conv1d(x_t, w_t)
123
124     x_np = x_t.numpy().ravel()
125     w_np = w_t.numpy().ravel()
126     # Valid cross-correlation in NumPy
127     y_np_corr = np.correlate(x_np, w_np, mode='valid')
128     # DSP convolution equals correlation with a flipped kernel.
129     y_np_conv_valid = np.convolve(x_np, w_np, mode='valid')
130
131     print('PyTorch conv1d (correlation):', y_corr.numpy().ravel())
132     print('NumPy correlate: ', y_np_corr)
133     print('NumPy DSP convolution valid: ', y_np_conv_valid)
134 except ImportError:

```

```

135     print('PyTorch is not installed in this environment; skip this optional cell.')
136
137
138 # FIR impulse response
139 h = np.array([0.25, 0.5, 0.25])
140 omega0 = 0.4*np.pi
141 n = np.arange(200)
142 x = np.exp(1j*omega0*n)
143
144 # For an FIR h[k], H(e^jw) is a finite sum.
145 k = np.arange(len(h))
146 H0 = np.sum(h * np.exp(-1j*omega0*k))
147
148 # Ignore startup/transient effects by using direct finite convolution and compare where
149 # all taps overlap.
150 y = np.convolve(x, h, mode='full')
151 y_pred = H0 * np.exp(1j*omega0*np.arange(len(y)))
152
153 # The eigenfunction relation is exact for an infinite-duration exponential.
154 # With our finite numerical record, compare the interior samples unaffected by record
155 # boundaries.
156 interior = slice(len(h)-1, len(x))
157 error = np.max(np.abs(y[interior] - y_pred[interior]))
158
159 print('H(e^jw0) =', H0)
160 print('|H| =', abs(H0))
161 print('phase =', np.angle(H0), 'rad')
162 print('max interior error =', error)
163
164
165 def dtft(x, n_index=None, omega=None):
166     x = np.asarray(x)
167     if n_index is None:
168         n_index = np.arange(len(x))
169     if omega is None:
170         omega = np.linspace(-np.pi, np.pi, 2048, endpoint=False)
171     E = np.exp(-1j*np.outer(omega, n_index))
172     return omega, E @ x
173
174 x = np.array([1.0, 2.0, -1.0])
175 omega, X = dtft(x)
176
177 fig = plt.figure(figsize=(9, 3))
178 plt.plot(omega/np.pi, np.abs(X))
179 plt.xlabel(r'$\omega/\pi$')
180 plt.ylabel(r'$|X(e^{j\omega})|$')
181 plt.title('DTFT magnitude over one 2π period')
182 plt.grid(True, alpha=0.25)
183 plt.show()
184
185 # Verify periodicity numerically at arbitrary points.
186 probe = np.array([-0.8, 0.2, 1.4])
187 _, X1 = dtft(x, omega=probe)
188 _, X2 = dtft(x, omega=probe + 2*np.pi)
189 print('max periodicity error:', np.max(np.abs(X1-X2)))
190
191
192 fig = plt.figure(figsize=(9, 4))
193 for a in [0.5, 0.9, 0.99]:
194     w, H = signal.freqz([1.0], [1.0, -a], worN=2048)
195     plt.plot(w/np.pi, np.abs(H), label=f'a={a}')
196 plt.xlabel(r'$\omega/\pi$')
197 plt.ylabel(r'$|H(e^{j\omega})|$')
198 plt.title('One-pole frequency response as the pole moves toward 1')
199 plt.legend()
200 plt.grid(True, alpha=0.25)
201 plt.show()

```

```

200
201
202 x = np.array([1.0, -2.0, 0.5, 1.25])
203 omega = np.linspace(-np.pi, np.pi, 200_000, endpoint=False)
204 _, X = dtfft(x, omega=omega)
205
206 E_time = np.sum(np.abs(x)**2)
207 E_freq = np.mean(np.abs(X)**2) # mean over one 2π period equals (1/2π) integral
208
209 print('time-domain energy      =', E_time)
210 print('frequency-domain approx =', E_freq)
211 print('absolute error          =', abs(E_time-E_freq))
212
213
214 # Simple edge/change detector on a 1-D signal
215 x = np.r_[np.zeros(20), np.ones(25), 0.3*np.ones(20), np.zeros(20)]
216 h = np.array([1.0, -1.0])
217 feature = np.convolve(x, h, mode='same')
218
219 fig = plt.figure(figsize=(9, 3))
220 plt.plot(x, label='signal')
221 plt.plot(feature, label='change-detector response')
222 plt.xlabel('n')
223 plt.title('A fixed convolution kernel as a feature detector')
224 plt.legend()
225 plt.grid(True, alpha=0.25)
226 plt.show()
227
228

```